
KyungDong University Global Campus



Smart Parking Monitoring and Availability Prediction System in KDU Global Campus

By

ALINGILYA CRISPIN NJEWA	2517116
Rai Chhenwi Hang	2517104
Agot John Ngor Majok	2517047
Hellah Audrey	2417263
Le Tri Thanh	2517105
Tharu Roshan	2517120

Supervisor

Dr. Ghulam Sarwar

Bachelor of Science in Smart Computing (Spring 2026)

DEPARTMENT OF SMART COMPUTING

KyungDong University Global Campus



Smart Parking Monitoring and Availability Prediction System in KDU Global Campus

A project presented to
KyungDong University Global

In partial fulfillment
of the requirement for the degree of

Bachelor of Science in Smart Computing (Spring 2026)

By

ALINGILYA CRISPIN NJEWA	2517116
Rai Chhenwi Hang	2517104
Agot John Ngor Majok	2517047
Hellah Audrey	2417263
Le Tri Thanh	2517105
Tharu Roshan	2517120

DEPARTMENT OF SMART COMPUTING

ABBREVIATIONS

SRS	Software Require Specification
PC	Personal Computer

TABLE OF CONTENTS

1	INTRODUCTION.....	52
1.1	Brief.....	52
1.2	Relevance to Course Modules.....	52
1.3	Project Background.....	52
1.4	Literature Review.....	52
1.5	Analysis from Literature Review (in the context of your project).....	52
1.6	Methodology and Software Lifecycle for This Project.....	52
1.6.1	The rationale behind Selected Methodology.....	52
2	PROBLEM DEFINITION	53
2.1	Problem Statement	53
2.2	Deliverables and Development Requirements.....	53
2.3	Current System (if applicable to your project).....	53
3	REQUIREMENT ANALYSIS.....	53
3.1	Use Cases Diagram(s)	54
3.2	Detailed Use Case	54
3.3	Functional Requirements.....	54
3.4	Non-Functional Requirements	54
4	DESIGN AND ARCHITECTURE	55
4.1	System Architecture	55
4.2	Data Representation [Diagram + Description].....	55
4.3	Process Flow/Representation	55
4.4	Design Models [along with descriptions].....	55
5	IMPLEMENTATION	56
5.1	Algorithm	56
5.2	External APIs	56
5.3	User Interface	56
6	TESTING AND EVALUATION	57
6.1	Manual Testing.....	57
6.1.1	System testing.....	57
6.1.2	Unit Testing	57
6.1.3	Functional Testing	58
6.1.4	Integration Testing.....	58
6.2	Automated Testing	59
	Tools used:	59
7	CONCLUSION AND FUTURE WORK	60
7.1	Conclusion.....	60
7.2	Future Work	60
8	REFERENCES.....	61

LIST OF FIGURES

Figure 1.1 Block Diagram.....	8
Figure 2.1 Use Case Diagram	9

LIST OF TABLES

Table 1.1 Block Diagram	8
Table 2.1 Use Case Diagram.....	9

INTRODUCTION

This chapter provides an overview of the project. The first paragraph of every chapter should provide the chapter summary.

Brief

This project presents a vision-based intelligent parking management system designed to detect, monitor, and predict parking space availability in real time within KDU Global Campus. The system leverages computer vision techniques such as object detection and machine learning models to analyze parking occupancy using camera input. The final outcome is a smart system capable of improving parking efficiency and reducing congestion through real-time monitoring and predictive analytics.

Relevance to Course Modules

This project is highly relevant to the following course modules:

- Computer Vision
- AI Security & Cryptography
- Data Structures and Algorithms

It integrates theoretical knowledge with practical implementation in real-world scenarios.

Project Background

Parking management has become an important issue in modern environments such as universities, smart cities, and commercial areas. Traditional parking systems often rely on manual observation or sensor-based technologies, which can be costly, difficult to maintain, and limited in scalability.

With the advancement of computer vision, it is now possible to automate parking monitoring using cameras. Computer vision is a field of Artificial Intelligence that enables machines to interpret and analyze visual data such as images and videos. By using techniques like object detection and image classification, systems can identify objects (e.g., vehicles) and understand their positions within a scene.

In the context of this project, computer vision is used to analyze video streams from cameras installed in parking areas. The system detects vehicles and determines whether a parking space is occupied or available. This is typically achieved using deep-learning models such as YOLO (You Only Look Once), which can perform real-time object detection with high accuracy.

The working principle of such a system involves capturing video input, processing frames, detecting vehicles, and mapping them to predefined parking slots. Based on this analysis, the system classifies each parking space as either free or occupied. Additionally, by storing historical data, machine learning techniques can be applied to predict future parking availability.

Existing applications such as KakaoMap and Kakao T provide parking location information, but they primarily rely on static or external sensors. They do not fully utilize camera-based real-time analysis, especially in small or unstructured environments like university campuses.

Therefore, this project is based on the idea of developing a vision-based intelligent parking system that combines real-time detection with predictive analytics. By leveraging computer vision and machine learning, the system aims to provide a more flexible, scalable, and cost-effective solution for parking management in localized environments such as KDU Global Campus.

Literature Review

Recent advancements in Computer Vision and deep learning have significantly improved the development of intelligent parking management systems. Many researchers have explored vision-based approaches to detect parking occupancy, replacing traditional sensor-based systems due to their flexibility and cost-effectiveness. Early approaches relied on classical image processing techniques such as edge detection and background subtraction to identify vehicles in parking spaces. However, these methods were limited in handling complex conditions such as lighting variations, occlusions, and weather changes.

With the emergence of deep learning, **Convolutional Neural Networks (CNNs)** have been widely used for parking occupancy classification. For instance, studies have proposed CNN-based models capable of classifying parking spots as occupied or vacant with high accuracy. These models are often trained on benchmark datasets such as PKLot and CNRPark, which provide labeled images under different environmental conditions.

More recently, real-time object detection models such as **YOLO (You Only Look Once)** have gained popularity due to their speed and efficiency. Research has demonstrated that YOLO-based systems can accurately detect vehicles and determine parking occupancy in real time, making them suitable for practical deployment. In addition, several studies have focused on improving model performance through architectural enhancements. For example, advanced models based on MobileNet and YOLO variants have achieved high accuracy (over 98%) in detecting parking occupancy while maintaining real-time performance. Another important research direction is the **automatic detection of parking spaces**. Instead of manually defining parking slots, some methods use clustering and spatial analysis techniques to automatically identify parking areas from images. This improves system adaptability but introduces challenges related to camera angles and environmental variations.

Despite these advancements, several limitations remain. A systematic review of parking detection systems highlights that many approaches are highly dependent on specific datasets and lack generalization across different environments. Additionally, most existing systems focus only on occupancy detection and do not incorporate predictive analytics or temporal analysis. Furthermore, challenges such as varying lighting conditions, weather changes, occlusion, and camera positioning continue to affect system performance. These issues indicate the need for more robust and adaptable solutions.

Analysis from Literature Review (in the context of your project)

The literature review highlights that significant progress has been made in parking occupancy detection using computer vision techniques, particularly with the adoption of Convolutional Neural Networks (CNNs) and real-time object detection models such as YOLO. These approaches have demonstrated high accuracy and efficiency, especially when evaluated on standard datasets like PKLot and CNRPark. However, a critical analysis

of existing studies reveals several limitations. Firstly, most research focuses primarily on **parking occupancy detection**, with limited attention given to **predictive analysis**. While current systems can determine whether a parking space is occupied or vacant, they often fail to provide insights into future availability, which is essential for improving user decision-making.

Secondly, many existing solutions rely heavily on **predefined datasets and controlled environments**, which restricts their ability to generalize to real-world scenarios. Variations in lighting, camera angles, and environmental conditions often reduce system performance when deployed outside laboratory settings. Secondly, many existing solutions rely heavily on **predefined datasets and controlled environments**, which restricts their ability to generalize to real-world scenarios. Variations in lighting, camera angles, and environmental conditions often reduce system performance when deployed outside laboratory settings.

Thirdly, although some systems attempt automatic parking space detection, a large number still depend on **manually defined parking slots**, which limits scalability and adaptability to new environments. Additionally, most research does not address **behavioral aspects of parking**, such as illegal parking or prolonged occupancy, which are important factors in real-world parking management systems.

In comparison, the proposed project extends beyond traditional approaches by introducing a **multi-layered intelligent system**. In addition to real-time parking occupancy detection using models such as YOLO, the system incorporates:

- **Predictive analytics**, enabling estimation of future parking availability based on historical data
- **Illegal parking detection**, identifying vehicles parked outside designated areas
- **Parking duration monitoring**, providing insights into space utilization and misuse
- **Localized deployment**, specifically designed for KDU Global Campus, ensuring practical applicability in real-world conditions

Methodology and Software Lifecycle for This Project

A This project adopts a structured methodology that combines **computer vision development processes** with a modern **Software Development Life Cycle (SDLC)** approach. The goal is to ensure systematic development, continuous improvement, and successful deployment of the intelligent parking system.

From a technical perspective, the methodology follows the standard pipeline used in computer vision projects:

1. Data Collection

Relevant data is gathered from both existing datasets (such as PKLot, CNRPark, and Kaggle datasets) and real-world images captured from KDU Global Campus parking areas. This ensures that the model is trained on diverse and realistic data.

2. Data Preprocessing and Annotation

The collected data is cleaned, resized, and annotated. Parking spaces and vehicles are labeled to prepare the dataset for training. Data augmentation techniques (e.g., rotation, brightness adjustment) are applied to improve model robustness.

3. Model Development

Deep learning models such as YOLOv5 or YOLOv8 are implemented for real-time vehicle detection, while Convolutional Neural Networks (CNNs) may be used for classification tasks. The model is trained using the prepared dataset.

4. Model Evaluation and Optimization

The trained model is evaluated using performance metrics such as accuracy, precision, recall, and mean Average Precision (mAP). Based on the results, the model is optimized to improve performance.

5. System Integration

The trained model is integrated into a complete system that processes real-time video input, detects parking occupancy, and provides outputs through a user interface.

6. Prediction Module Development

A machine learning component is added to analyze historical data and predict future parking availability, enhancing the system's intelligence.

7. Deployment and Testing

The system is deployed in a simulated or real campus environment and tested to ensure reliability, efficiency, and usability.

The rationale behind Selected Methodology

The selection of methodology and software development life cycle (SDLC) for this project is driven by the nature of the problem, which involves both **software system development** and **machine learning-based computer vision tasks**.

Firstly, an **object-oriented approach** is adopted for system design and implementation. This approach allows the system to be structured into modular and reusable components such as data processing, vehicle detection, occupancy classification, prediction module, and user interface. Object-oriented design improves code maintainability, scalability, and ease of integration, which are essential for complex systems involving multiple functionalities.

Secondly, the project follows an **Agile-based SDLC model**. Agile is selected because this project involves iterative experimentation, especially in training and optimizing machine learning models. Unlike traditional models such as Waterfall, where each phase is completed sequentially, Agile allows continuous development, testing, and refinement.

This is particularly important because:

- Model performance depends on data quality and may require multiple training iterations

- System requirements may evolve during development
- Continuous testing is needed to ensure real-time performance and accuracy

Additionally, the integration of real-world data from KDU Global Campus introduces variability in lighting conditions, camera angles, and environmental factors. Agile methodology provides the flexibility needed to adapt to these changes and improve the system incrementally.

Furthermore, the computer vision development process itself naturally follows an iterative cycle of data collection, preprocessing, training, evaluation, and optimization. This aligns well with Agile principles, making it a suitable choice for this project.

The rationale behind Selected Methodology

The choice of methodology and software development life cycle for this project is guided by the *combined nature of software engineering and machine learning tasks*.

An *object-oriented design approach* is adopted to organize the system into modular components, including data acquisition, vehicle detection, occupancy classification, prediction, and user interface. This structure improves *maintainability and scalability*, allowing each module to be developed and updated independently.

The project follows an *Agile-based Software Development Life Cycle (SDLC)* to support *iterative development*. This is particularly suitable for computer vision applications, where model performance depends on multiple cycles of data preprocessing, training, and evaluation. Agile enables gradual refinement of both the model and the overall system without requiring a fixed sequence of stages.

Additionally, the use of real-world campus data introduces variability such as changes in lighting and camera perspectives. The selected methodology allows the system to be continuously adjusted to handle these variations effectively.

Overall, the combination of an *object-oriented design* and an *Agile SDLC model* provides a *flexible and structured framework*, ensuring efficient development and continuous improvement of the proposed system.

PROBLEM DEFINITION

This chapter discusses the precise problem to be solved. It should extend to include the outcome.

Problem Statement

The problem statement should be included here.

Deliverables and Development Requirements

Deliverables and development requirements.

Current System (if applicable to your project)

A brief description of an existing system.

The following figure is a sample figure, Figure 2.1. You are required to follow the same style of numbering and caption for the whole report.

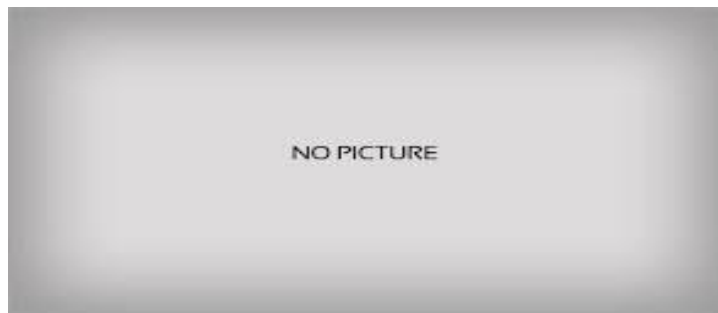


Figure 2.1: Sample picture

The following table (Table 2.1) is a sample table; you are required to follow the same style of numbering and caption for the whole report.

Table 1sample

Header 1	Header 2	Header 3
Text	Text	Text

The following list style is the sample to consistently follow in the whole report.

- List items 1
- List items 2

REQUIREMENT ANALYSIS

The following parts of the Software Requirements Specification (SRS) report should be included in this chapter.

Use Cases

Diagram(s) Detailed

Use Case Functional

Requirements

Non-Functional Requirements

DESIGN AND ARCHITECTURE

The following parts of the Software Design Description (SDD) report should be included in this chapter.

System Architecture

Data Representation [Diagram + Description]

Process Flow/Representation

Design Models [along with descriptions]

IMPLEMENTATION

This chapter will discuss implementation details supported by UML diagrams (if applicable). You will not put your source code here. Any of the following sections may be included based on your project.

Algorithm

Mention the algorithm(s) used in your project to get the work done with regard to major modules. Provide a pseudocode **OR** a natural language explanation regarding the functioning of main features. Be sure to use the correct syntax and semantics for algorithm representations.

External APIs

Describe the APIs used in the following table.

Table 8 Details of APIs used in the project

Name of API	Description of API	Purpose of usage	List down the function/ class name in which it is used

User Interface

Details about user interface with descriptions.

TESTING AND EVALUATION

This chapter may include the following sections. (Students are required to perform the testing both manually and automatedly).

Manual Testing

System testing

Once the system has been successfully developed, testing has to be performed to ensure that the system working as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing will help in finding the errors that may be hidden from the user. There are a few types of testing which include unit testing, functional testing, and integration testing. The testing must be completed before it is being deployed for a user to use.

Unit Testing

Unit Testing 1: Login as FYP Committee

Testing Objective: To ensure the login form is working correctly

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify user login after a click on the “Login” button on the login form with correct input data	Username:L001 Password:1234	Successfully log into the main page of the system as an FYP Committee member.	Pass
2.				

Unit Testing 2: Edit Profile

Testing Objective: To ensure the edit profile form is working properly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.				
2.				

Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each module is tested. This is to ensure that the system produced meets the specifications and requirements.

Functional Testing 1: Login with different roles

Objective: To ensure that the correct page with the correct navigation bar is loaded.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Login as an “FYP Committee” member.	Username: L001 Password: 1234	The main page for the FYP Committee member is loaded with the FYP committee navigation bar	Pass
2.				

Integration Testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Login as “FYP Committee” member	Username: L001 Password: 1234	Login successful and the FYP Committee page with its navigation bar is loaded and in the view profile page	Pass
2.	Upload student record for Project 1	-	File successfully uploaded and return to the upload page. Student records are updated.	Pass
3.	View supervising student	-	The list of supervisees shown on the screen.	Pass

Automated Testing:

Tools used:

Tool Name	Tool Description	Applied on [list of related test cases/FR/NFR]	Results

CONCLUSION AND FUTURE WORK

This chapter concludes the project and highlights future work. Future work must be included in the thesis.

Conclusion

Future Work

REFERENCES

References to any book, journal paper or website should properly be acknowledged. Please consistently follow the style. The following are few examples of different resources i.e., journal article, book, and website.

- [1] Lyda M.S. Lau, Jayne Curson, Richard Drew, Peter Dew and Christine Leigh, (1999), Use of VSP Resource Rooms to Support Group Work in a Learning Environment, ACM 99, pp-2. **(Journal paper example)**
- [2] Hideyuki Nakanishi, Chikara Yoshida, Toshikazu Nishimura and Tetsu Ishida, (1996), FreeWalk: Supporting Casual Meetings in a Network, pp 308-314 **(paper on web)**
<http://www.acm.org/pubs/articles/proceedings/cscw/240080/p308-nakanishi.pdf>
- [3] Ali Behforooz & Frederick J. Hudson, (1996), Software Engineering Fundamentals, Oxford University Press. Chapter 8, pp255-235. **(book reference example)**

